Module 2 QB SE Solved

1. Define Requirements Engineering. Explain the major activities involved in the Requirements Engineering process.

Definition

Requirements Engineering is the process of collecting software requirements from the client, understanding, evaluating, and documenting them.

It constructs a bridge between design and construction by ensuring that the software requirements are clearly understood before development begins.

Major Activities of Requirements Engineering

According to the Module 2 notes, the major activities involved in the Requirements Engineering process are as follows:

1. Inception

- Inception establishes a basic understanding of the problem to be solved.
- It identifies the people who want a solution and the nature of the desired solution.
- It also examines the effectiveness of preliminary communication and collaboration between stakeholders and the software team.

2. Elicitation

- Elicitation is the process of gathering requirements from customers, users, and other stakeholders.
- It identifies the objectives of the system, what must be accomplished, and how the system supports business needs.
- It also determines how the system or product will be used in day-to-day activities.

3. Elaboration

- During elaboration, the information collected during inception and elicitation is expanded and refined.
- It focuses on developing a refined requirements model.
- The model identifies different aspects of software function, behaviour, and information.

4. Negotiation

- Customers and users may request more features than can be achieved within the available budget and time.
- Different stakeholders may also propose conflicting requirements.
- Negotiation helps resolve these conflicts and establish realistic requirements that satisfy the majority of stakeholder needs within the available resources.
- The goal is to achieve a win-win result, where stakeholders receive a useful system and the software team works within achievable budgets and deadlines.

5. Specification

- Specification is the process of documenting the software requirements clearly.
- A specification can take different forms, such as:
    - A written document.
    - A set of graphical models.
    - A formal mathematical model.
    - A collection of usage scenarios.
    - A prototype or a combination of these.
- It provides a clear description of what the software must do.

6. Validation

- Validation checks the quality and correctness of the requirements.
- It ensures that all requirements are stated unambiguously.
- It identifies and corrects inconsistencies, omissions, and errors.
- It also ensures that the requirements conform to the standards established for the process, project, and product.

7. Requirements Management

- Requirements Management is a set of activities used to identify, control, and track requirements and changes to requirements throughout the project.
- Since requirements may change during the life of a system, this activity helps the project team manage those changes systematically.
- It ensures that changes are tracked as the project progresses.

Conclusion

Requirements Engineering is an important software engineering process that helps understand, document, validate, and manage customer requirements. Its seven major activities help ensure that the software developed meets stakeholder needs within the available time and budget.

2. List and explain the activities involved in Establishing the Groundwork for Requirements Engineering.

Establishing the groundwork is an important activity in Requirements Engineering. It helps stakeholders and software engineers understand the problem, identify the requirements, and work together to develop the desired software system.

According to the Module 2 notes, the following four activities are involved in establishing the groundwork.

1. Identifying Stakeholders

- Stakeholders are the people who have an interest in the software system being developed.
- Each stakeholder has a different view of the system and expects different benefits from its successful development.
- They may also face different risks if the development project fails.
- Identifying stakeholders helps the software team understand their needs and expectations.

2. Recognizing Multiple Viewpoints

Different stakeholders have different expectations from the software system.

- Business managers: Interested in features that can be developed within the budget and delivered within the required market time.
- End users: Prefer familiar features that are easy to learn and use.
- Software engineers: Focus on technical functions and infrastructure that support the required features.
- Support engineers: Focus on the maintainability of the software.

Recognizing these viewpoints helps the team understand different requirements and avoid overlooking important needs.

3. Working Toward Collaboration

- Collaboration involves stakeholders working together and providing their views about the requirements.
- It does not necessarily mean that all requirements must be decided by a committee.
- Stakeholders share their needs and expectations to help define the requirements.
- A strong project champion, such as a business manager or senior technologist, may make the final decision about which requirements should be included.

Collaboration improves communication and helps the team reach decisions about the software requirements.

4. Asking the First Questions

- At the beginning of a project, the software team should ask context-free questions.
- These questions focus on the customer, other stakeholders, overall project goals, and expected benefits.
- They help the team understand the purpose of the project before focusing on technical details.

Examples of context-free questions include:

- Who is behind the request for this work?
- Who will use the solution?
- What will be the economic benefit of a successful solution?
- Is there another source for the solution that is needed?

These questions help identify all stakeholders, measure the benefits of successful implementation, and consider possible alternatives to custom software development.

Conclusion

Establishing the groundwork involves identifying stakeholders, recognizing multiple viewpoints, working toward collaboration, and asking the first questions. These activities create a common understanding of the project and provide a strong foundation for gathering and defining software requirements.

3. How can you develop an effective use case? Develop a UML use case diagram for Home Security Function.

1. Developing an Effective Use Case

A use case describes the interactions between an actor and a computer-based system to accomplish a specific goal. It captures the system's functionality from the user's point of view.

An effective use case can be developed through the following steps.

Step 1: Identify the actors

- Identify the people, devices, or external systems that interact with the software.
- For the Safe Home system, the actors include the homeowner, setup manager, sensors, and monitoring and response subsystem.

Step 2: Identify the user's goals

- Determine what each actor wants to achieve using the system.
- For example, the homeowner may want to activate or deactivate the security system, access camera views, or respond to an alarm event.

Step 3: Create a preliminary use case

- Write a simple description of how the actor interacts with the system to achieve a goal.
- Identify the main actions performed by the actor and the responses provided by the system.

Step 4: Refine the preliminary use case

Examine each step of the primary scenario by asking the following questions:

- Can the actor take some other action at this point?
- Is it possible that the actor will encounter an error condition? If so, what might it be?
- Is it possible that the actor will encounter other behaviour caused by an event outside the actor's control?

This helps identify alternative interactions, error conditions, and exceptional situations.

Step 5: Write a formal use case

A formal use case includes the following elements:

- Goal in context: Describes the overall purpose and scope of the use case.
- Precondition: Describes what must be true before the use case begins.
- Trigger: Identifies the event or condition that starts the use case.
- Scenario: Lists the actions performed by the actor and the corresponding system responses.

These steps help develop a complete use case that clearly describes the system's behaviour.

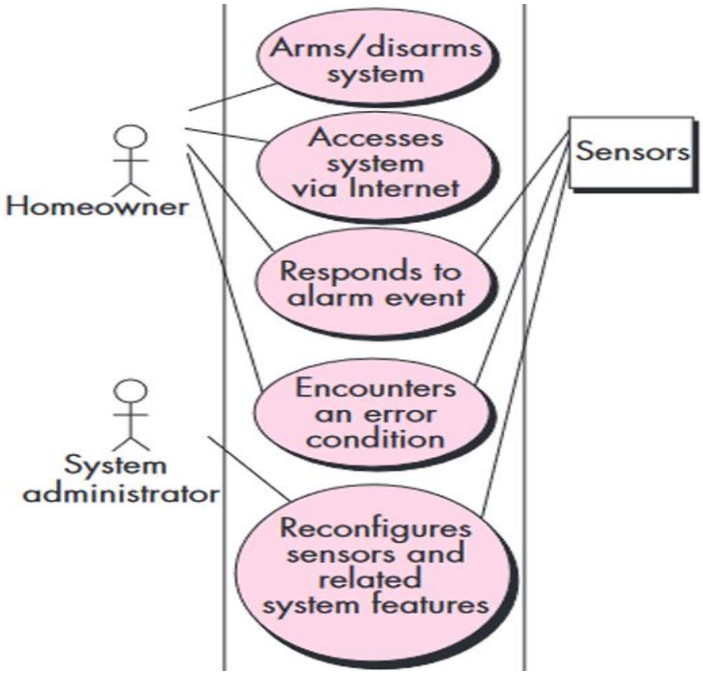
2. UML Use Case Diagram for Home Security Function

The Safe Home system is a home security system that allows the homeowner to monitor and control security functions. The UML use case diagram represents the interaction between actors and the home security system.

Actors identified in the notes:

- Homeowner: Interacts with the system to control security functions.
- System Administrator: Reconfigures sensors and related system features.
- Sensors: Provide information about security events.

Diagram to draw in the examination



3. Explanation of the Diagram

1. Arms/disarms system: The homeowner activates or deactivates the security system.
2. Accesses system via Internet: The homeowner accesses the security system remotely.
3. Responds to alarm event: The system handles an alarm event generated by the security system.
4. Encounters an error condition: The system deals with an error condition.
5. Reconfigures sensors and related system features: The system administrator changes sensor configurations and related features.

Conclusion

An effective use case identifies actors, their goals, the primary scenario, alternative interactions, and possible error conditions. The UML use case diagram for the SafeHome system provides a high-level view of the home security functions and the interactions between the homeowner, system administrator, sensors, and the system.

4. Explain the process of Building the Requirements Model. Discuss the different elements represented in the model.

1. Introduction

The Requirements Model is an analysis model that describes the required informational, functional, and behavioral aspects of a computer-based system.

It helps software engineers understand what the customer requires and provides a basis for creating the software design. The model changes dynamically as the development team learns more about the system and stakeholders understand their requirements more clearly.

2. Process of Building the Requirements Model

The process involves analyzing and representing the requirements from different viewpoints.

Step 1: Understand the requirements

- Collect and study the requirements obtained during inception, elicitation, and negotiation.
- Understand the problem domain and the expected functionality of the system.

Step 2: Develop scenario-based elements

- Describe the system from the user's point of view.
- Identify actors, their goals, and their interactions with the system.
- Develop use cases and use-case diagrams to represent the scenarios.

Step 3: Develop class-based elements

- Identify the objects and classes involved in the system.
- Determine their attributes, operations, and relationships.
- Represent the important classes using class diagrams.

Step 4: Develop behavioral elements

- Study how the system behaves in response to events.
- Identify the different states of the system and the events that cause transitions between states.
- Use behavioral models, such as UML state diagrams, to represent this behaviour.

Step 5: Review and refine the model

- Check whether the model correctly represents the requirements.
- Refine the model as additional information becomes available.
- Ensure that the model provides a clear understanding of the system to be developed.

3. Different Elements Represented in the Requirements Model

According to the notes, the requirements model consists of the following elements.

1. Scenario-Based Elements

- The system is described from the user's point of view using a scenario-based approach.
- These elements explain how users interact with the system to accomplish specific goals.
- Basic use cases and their corresponding use-case diagrams can be developed into more detailed, template-based use cases.

- UML activity diagrams can also be used to represent the flow of activities involved in a scenario.

Example: In the SafeHome security system, a homeowner activates the security system by entering a password and selecting the required mode.

2. Class-Based Elements

- Each usage scenario involves a set of objects that are manipulated when an actor interacts with the system.
- These objects are categorized into classes.
- A class is a collection of things that have similar attributes and common behaviours.
- Class-based elements represent the attributes and operations associated with each class.
- UML class diagrams are used to represent these elements.

Example: In the SafeHome system, a Sensor class may have attributes such as name and type, and operations such as identify and enable.

3. Behavioral Elements

- The behaviour of a computer-based system influences the software design and implementation approach.
- Therefore, the requirements model must include elements that represent the system's behaviour.
- These elements describe the different states of the system and the events that cause it to change from one state to another.
- UML state diagrams are used to represent system behaviour and the actions performed in response to particular events.

Example: In the SafeHome system, the control panel changes its state depending on user input, such as entering a password or activating the security system.

4. Flow-Oriented Elements

- Information is transformed as it flows through a computer-based system.
- The system accepts input in different forms, applies functions to transform the information, and produces output.
- Inputs may include control signals, numbers entered by a user, network packets, or data files.
- Flow-oriented elements help represent how data moves through the system and how it is transformed.

Example: In a banking system, account details are accepted as input, processed to perform a transaction, and used to produce an updated account balance.

5. Analysis Patterns

- Analysis patterns suggest reusable solutions, such as a class, function, or behaviour, within an application domain.

- They help speed up the development of abstract analysis models that capture the main requirements of a problem.
- They provide reusable models, examples, advantages, and limitations.
- They also facilitate the transformation of an analysis model into a design model by suggesting design patterns and reliable solutions to common problems.

Example: A reusable account-handling pattern can help model similar account-related requirements in different banking applications.

Conclusion

Building the Requirements Model helps describe the informational, functional, and behavioral requirements of a software system. Scenario-based, class-based, behavioral, and flow-oriented elements, together with analysis patterns, help the development team understand the requirements and establish a foundation for software design.

5. Discuss the process of Negotiating and Validating Requirements. Why is Requirements Negotiation Necessary?

1. Negotiating Requirements

Requirements negotiation is the process of resolving conflicts between customer requirements and stakeholder expectations to achieve a win-win result within realistic budgets and deadlines.

Process of Negotiation

According to Boehm, the following activities are performed:

1. Identification of stakeholders: Identify the key stakeholders involved in the system or subsystem.

2. Determination of win conditions: Identify the conditions that each stakeholder considers necessary for project success.

3. Negotiation of win conditions: Resolve conflicting requirements and establish mutually acceptable conditions for all stakeholders, including the software team.

2. Validating Requirements

Requirements validation is the process of examining the requirements model to ensure that the requirements are correctly specified and meet the project's objectives.

The following checks are performed during validation:

1. Consistency: Requirements must be consistent with the overall system objectives.
2. Necessity: Each requirement must be necessary for the system.
3. Clarity: Requirements must be clearly defined and unambiguous.
4. Conflict detection: Requirements must not conflict with one another.
5. Feasibility: Requirements must be achievable in the technical environment.
6. Testability: Each requirement must be testable after implementation.
7. Completeness: The model must correctly represent the information, functions, and behaviour of the system.

3. Why Is Requirements Negotiation Necessary?

- Customers may request more features than the available budget and time permit.
- Different stakeholders may propose conflicting requirements.
- It helps prioritize essential requirements and resolve disagreements.
- It ensures that requirements are realistic and achievable.
- It helps satisfy stakeholder needs while meeting project budgets and deadlines.

Conclusion

Requirements negotiation resolves conflicts and establishes realistic requirements, while validation checks their correctness, clarity, consistency, and feasibility. Both activities help ensure successful software development.

6. Explain Scenario-Based Modeling. Describe how scenarios are used during Requirements Analysis.

1. Scenario-Based Modeling

Scenario-based modeling is a requirements analysis technique in which the system is described from the user's point of view. It explains how an actor interacts with the system to achieve a specific goal.

A use case captures the interactions between the users, producers and consumers of information, and the system itself.

2. Steps in Scenario-Based Modeling

1. Creating a Preliminary Use Case

- Identify the actor and the goal to be achieved.
- Write a simple description of how the actor interacts with the system.

2. Refining a Preliminary Use Case

- Examine each step of the primary scenario.
- Identify alternative actions, possible error conditions, and other behaviours caused by external events.

3. Writing a Formal Use Case

- Document the use case in a structured format.
- Include the goal in context, preconditions, trigger, scenario steps, and system responses.

3. How Scenarios Are Used During Requirements Analysis

- Understanding user requirements: Scenarios explain the system's functions from the user's perspective.
- Identifying interactions: They describe how actors interact with the system to achieve their goals.
- Identifying exceptions: They help identify alternative flows and error conditions.
- Clarifying system behaviour: They describe the actions performed by the actor and the corresponding system responses.
- Supporting UML models: Use cases and UML activity diagrams represent the interactions and flow of activities in a scenario.

Example: In the SafeHome system, a homeowner enters a password and selects the STAY or AWAY option to activate the security system.

Conclusion

Scenario-based modeling helps developers understand user requirements, identify system interactions, and describe expected behaviour. It provides a foundation for developing use cases and other requirements models.

7. Demonstrate Access Camera Surveillance via Internet – Display Camera Views Function with a Neat Activity Diagram.

1. Description

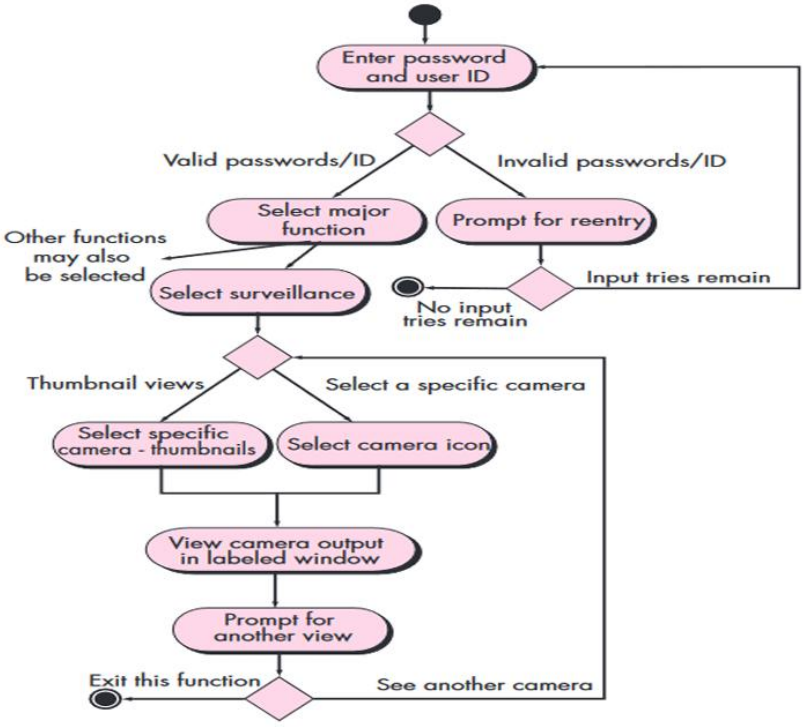
Access Camera Surveillance via the Internet – Display Camera Views (ACS-DCV) is a use case of the SafeHome security system. It describes how a homeowner accesses the security system through the Internet to view camera feeds.

2. Steps Involved

1. The homeowner accesses the SafeHome security system through the Internet.
2. The system verifies the homeowner's access.
3. The homeowner selects the required camera.
4. The system retrieves and displays the camera view.
5. The homeowner can select another camera or end the session.

These steps illustrate the basic flow of interaction. The detailed ACS-DCV example is referred to the textbook in the provided notes.

3. UML Activity Diagram



Explain the diagram a bit

8. Describe the Data Modeling Concepts and identify data objects, attributes and relationships with a suitable example.

1. Data Modeling Concepts

Data modeling is the process of identifying data objects, defining their attributes, and establishing the relationships between them. It helps software engineers understand how information is organized and how data is connected within a system.

According to the notes, data objects are modeled by defining their attributes and relationships. The processes that manipulate these data objects are modeled separately to show how data is transformed as it flows through the system.

The main concepts of data modeling are:

1. Data Objects

A data object represents information that must be understood and processed by the software system. It represents an entity or concept about which the system stores information.

Each data object has its own properties and may be connected to other data objects through relationships.

Example: In a Student Management System, Student and Course are data objects. The system stores information about students and the courses they study.

2. Attributes

Attributes are the properties or characteristics that describe a data object. They provide specific details about the object and help distinguish one object from another.

Each data object contains a set of attributes relevant to its purpose. The values of these attributes describe a particular instance of the object.

Example: A Student object may have Student ID, Name, Age, and Email as its attributes. For a particular student, these attributes might contain the values 101, Ravi, 20, and ravi@example.com.

3. Relationships

Relationships describe the connections between two or more data objects. They explain how the objects are associated with each other in the system.

Relationships are important because data objects generally do not exist independently. They interact with or depend on other objects to perform the required functions.

Example: In a Student Management System, a Student enrolls in a Course. This relationship connects the Student object with the Course object.

2. Suitable Example: Student Management System

Consider a college software system that maintains student details and course information.

- Data objects: Student and Course represent the main entities about which the system stores information.

- Attributes: Student has attributes such as Student ID, Name, Age, and Email. Course has attributes such as Course ID, Course Name, and Credits.
- Relationship: A Student enrolls in a Course. This relationship connects the student information with the course information.

For example, student Ravi, whose Student ID is 101, enrolls in the course Computer Science. The student's details are stored in the Student object, the course details are stored in the Course object, and the enrollment relationship connects them.

Thus, data objects identify what information is stored, attributes describe the properties of that information, and relationships explain how different data objects are connected.

Conclusion

Data modeling is an important part of requirements analysis. It identifies the data objects, their attributes, and their relationships. These concepts help organize information clearly and provide a foundation for designing the software system.

9. Demonstrate Class-Based Modeling and Identify the Analysis Classes

1. Class-Based Modeling

Class-based modeling is a requirements analysis technique that identifies the classes involved in a system and describes their attributes, operations, and relationships.

According to the Module 2 notes, each usage scenario implies a set of objects that are manipulated when an actor interacts with the system. These objects are categorized into classes based on their similar attributes and common behaviours.

A UML class diagram is used to represent the classes and their attributes and operations.

2. Identifying Analysis Classes

Analysis classes are identified by examining the requirements and usage scenarios to determine the important objects involved in the system.

The following steps are used:

1. Study the requirements: Understand the system requirements and use cases.
2. Identify objects: Find the objects involved when an actor interacts with the system.
3. Group objects into classes: Categorize objects having similar attributes and common behaviours.
4. Identify attributes: Determine the information that each class must maintain.

5. Identify operations: Determine the actions that can be performed on the objects.
6. Identify relationships: Determine how the classes are connected and interact with one another.

3. Example: Sensor Class in SafeHome Security System

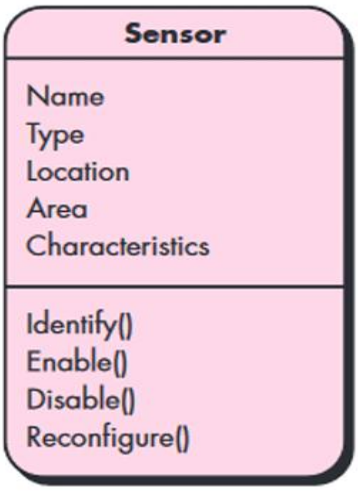
In the SafeHome security system, the Sensor class represents the sensors used to monitor the home.

The class contains the following attributes and operations:

- Attributes: Name, Type, Location, Area, Characteristics.
- Operations: Identify(), Enable(), Disable(), Reconfigure().

The attributes describe the sensor, while the operations define the actions that can be performed on it.

UML Class Diagram



Conclusion

Class-based modeling helps identify the important classes involved in a system and specify their attributes and operations. In the SafeHome system, the Sensor class represents the sensor information and the operations required to control it.

10. Using CRC Modeling, Draw a Class Diagram and CRC Index Card for the Floorplan

I'll give you the written content separately so you can find and copy the diagrams from your textbook yourself. I won't shorten the answer just because a diagram is required.

1. CRC Modeling

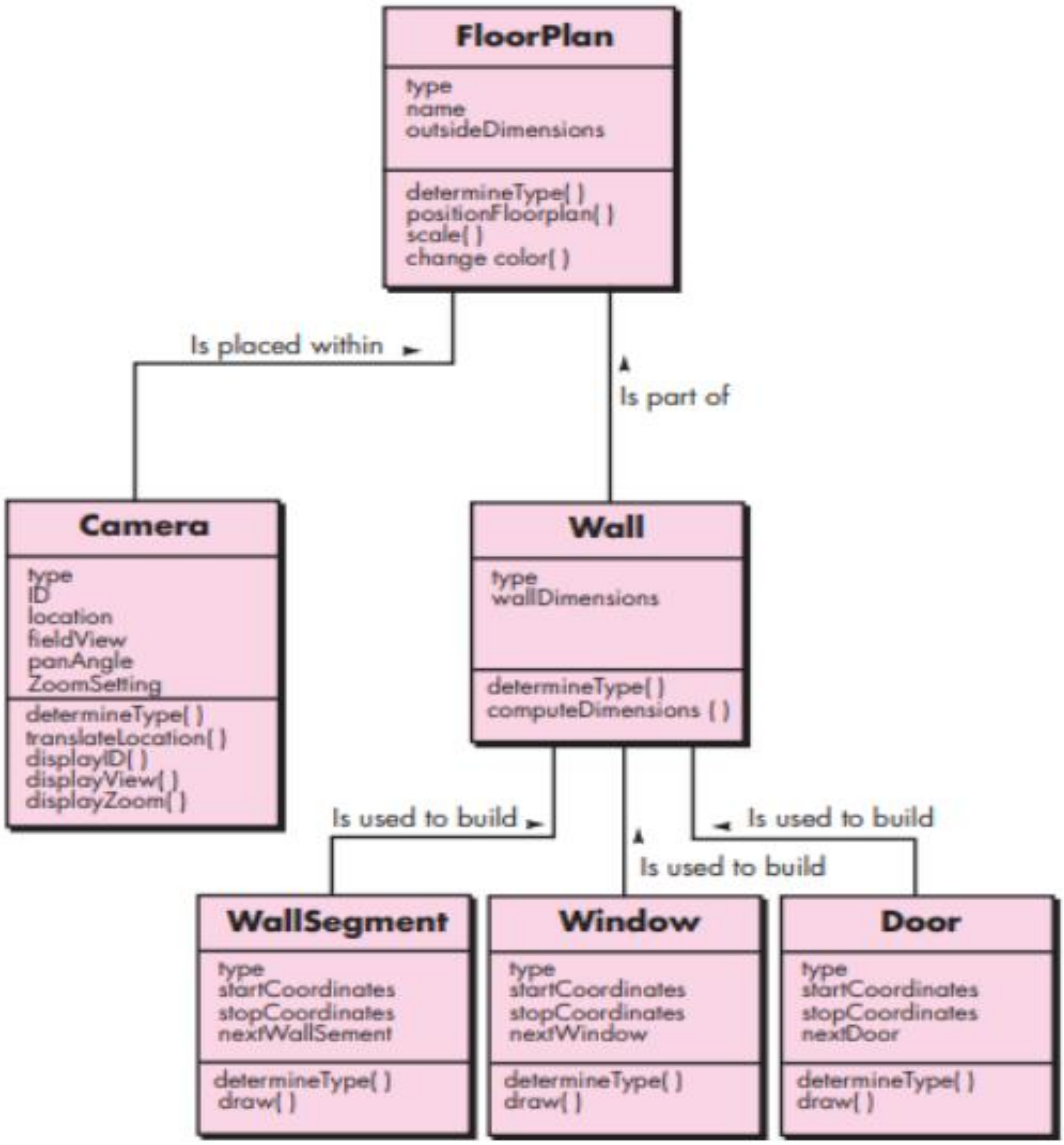
CRC stands for Class–Responsibility–Collaborator. CRC modeling is a technique used in class-based modeling to identify classes, their responsibilities, and the other classes with which they collaborate.

- Class: Represents an object or entity in the system.
- Responsibility: Describes the tasks or functions performed by the class.
- Collaborator: Identifies other classes that help a class perform its responsibilities.

CRC modeling helps understand how different classes work together to satisfy the requirements of a software system.

2. Class Diagram for FloorPlan

The FloorPlan class diagram represents the floorplan and its relationships with other classes in the SafeHome system. The following details are from the textbook reference, since this diagram is not included in your notes.



Classes, attributes and operations

1. FloorPlan

- Attributes: `type`, `name`, `outsideDimensions`
- Operations: `determineType()`, `positionFloorplan()`, `scale()`, `changeColor()`

2. Camera

- Attributes: `type`, `ID`, `location`, `fieldView`, `panAngle`, `zoomSetting`
- Operations: `determineType()`, `translateLocation()`, `displayID()`, `displayView()`, `displayZoom()`

3. Wall

- Attributes: `type`, `wallDimensions`
- Operations: `determineType()`, `computeDimensions()`

4. WallSegment

- Attributes: `type`, `startCoordinates`, `stopCoordinates`, `nextWallSegment`
- Operations: `determineType()`, `draw()`

5. Window

- Attributes: `type`, `startCoordinates`, `stopCoordinates`, `nextWindow`
- Operations: `determineType()`, `draw()`

6. Door

- Attributes: `type`, `startCoordinates`, `stopCoordinates`, `nextDoor`
- Operations: `determineType()`, `draw()`

Relationships between classes

- Camera is placed within FloorPlan.
- Wall is part of FloorPlan.
- WallSegment, Window and Door are used to build a Wall.

Use these class names, attributes, operations and relationships when drawing the UML class diagram.

3. CRC Index Card for FloorPlan

CRC stands for Class–Responsibility–Collaborator. The CRC index card identifies the responsibilities of the FloorPlan class and the classes that collaborate with it.

Class: FloorPlan

Responsibility	Collaborator
Defines floor plan name/type	—
Manages floor plan positioning	—
Scales floor plan for display	—
Incorporates walls, doors and windows	Wall
Shows position of video cameras	Camera

The first three responsibilities are handled by the FloorPlan class itself. The other two require collaboration with the Wall and Camera classes.

Conclusion

The FloorPlan class diagram represents the structure of the floorplan and its associated classes, while the CRC index card describes its responsibilities and collaborators. Together, they help identify class behaviour and interactions during requirements analysis.

11. Explain the three types of QFD with examples.

Quality Function Deployment (QFD) is a quality management technique that translates customer needs into technical requirements for software. It concentrates on maximizing customer satisfaction throughout the software engineering process by understanding what is valuable to customers and deploying these values throughout development.

QFD identifies three types of requirements:

1. Normal Requirements

Normal requirements are the features and functions that customers explicitly request. Customer satisfaction increases when these requirements are properly implemented.

Examples:

- Different types of graphical displays.

- Specific system functions.
- Defined levels of performance.

Software example: In a banking application, customers request balance checking, fund transfers and transaction history. These are normal requirements because customers directly expect these functions to be provided.

2. Expected Requirements

Expected requirements are features that customers assume will be present in the software, even if they do not explicitly mention them. If these requirements are missing, customers become dissatisfied.

Examples:

- Ease of human–machine interaction.
- Overall operational correctness and reliability.
- Ease of software installation.

Software example: In a banking application, customers expect secure transactions, correct account balances, reliable operation and an easy-to-use interface. If the application frequently crashes or displays incorrect balances, customers will be dissatisfied.

3. Exciting Requirements

Exciting requirements are unexpected features or capabilities that customers do not necessarily anticipate but that provide additional satisfaction when available. These features can make a product more attractive than competing products.

Examples:

- Multitouch screens.
- Visual voicemail.
- Unexpected capabilities that delight users.

Software example: A mobile application may provide standard features such as messaging and calling, along with an innovative feature such as automatically converting voice messages into text. Customers may not have expected this feature, but its availability improves their experience.

Conclusion

QFD helps software developers understand and prioritize customer needs. Normal requirements provide requested functionality, expected requirements ensure basic quality and reliability, and exciting requirements provide unexpected benefits that increase customer satisfaction.

12. Case Study 1: Library Management System

Question: Identify and explain the functional and non-functional requirements for the Library Management System.

A Library Management System is a software application used to manage books, library members, book issuing, returns and other library activities. Its requirements are classified into functional requirements and non-functional requirements.

1. Functional Requirements

Functional requirements describe what the system should do. They specify the services and operations provided by the software.

1. Book Management: The system should allow the librarian to add, update, delete and search for books using details such as book title, author, ISBN and category.

2. Member Management: The system should maintain member details such as member ID, name, contact information and membership status.

3. Book Issue: The system should allow the librarian to issue available books to registered members and record the issue date and due date.

4. Book Return: The system should record returned books, update their availability and calculate fines for overdue returns.

5. Book Search: Members and librarians should be able to search for books by title, author, ISBN or category and check their availability.

6. Fine Management: The system should calculate overdue fines according to library rules and maintain fine payment details.

7. Report Generation: The system should generate reports about available books, issued books, overdue books, registered members and collected fines.

2. Non-Functional Requirements

Non-functional requirements specify how well the system should operate rather than the specific functions it performs. They define the quality attributes and constraints of the system.

1. Performance: The system should retrieve book details and process issue or return transactions quickly, even when many users access it simultaneously.

2. Reliability: The system should operate correctly and consistently, maintaining accurate records of books, members, issues and returns.

3. Security: Only authorized users should be allowed to perform administrative operations. Member information should be protected through authentication and access control.

4. Usability: The system should have a simple, user-friendly interface so that librarians and members can use it without difficulty.

5. Availability: The system should be available during library working hours, allowing users to search for books and librarians to manage transactions.

6. Maintainability: The software should be easy to update when library rules change or new features, such as online reservations, are introduced.

7. Data Integrity: The system should prevent inconsistent records, such as issuing an unavailable book or recording the same return more than once.

Conclusion

Functional requirements define the operations of the Library Management System, such as book searching, issuing and returning. Non-functional requirements define its quality, including performance, reliability, security and usability.

13. Case Study 2: Library Management System

Question: Demonstrate UML modelling using a use case diagram. Identify and explain the list of actors and use cases for a Library Management System.

1. Introduction

A Use Case Diagram is a UML (Unified Modeling Language) diagram that represents the interaction between external users (actors) and the functions provided by a system. It describes the system's functional requirements from the user's point of view.

For a Library Management System, the diagram shows how librarians, library members and administrators interact with the system to manage books and library activities.

2. List of Actors

An actor is a person or external system that interacts with the software to achieve a particular goal.

1. Librarian: Manages books, issues books to members, accepts returned books, calculates fines and maintains library records.

2. Library Member: Searches for books, checks book availability, borrows books, returns books and views borrowing details.

3. Administrator: Manages user accounts, controls access permissions and maintains the overall system.

3. List of Use Cases

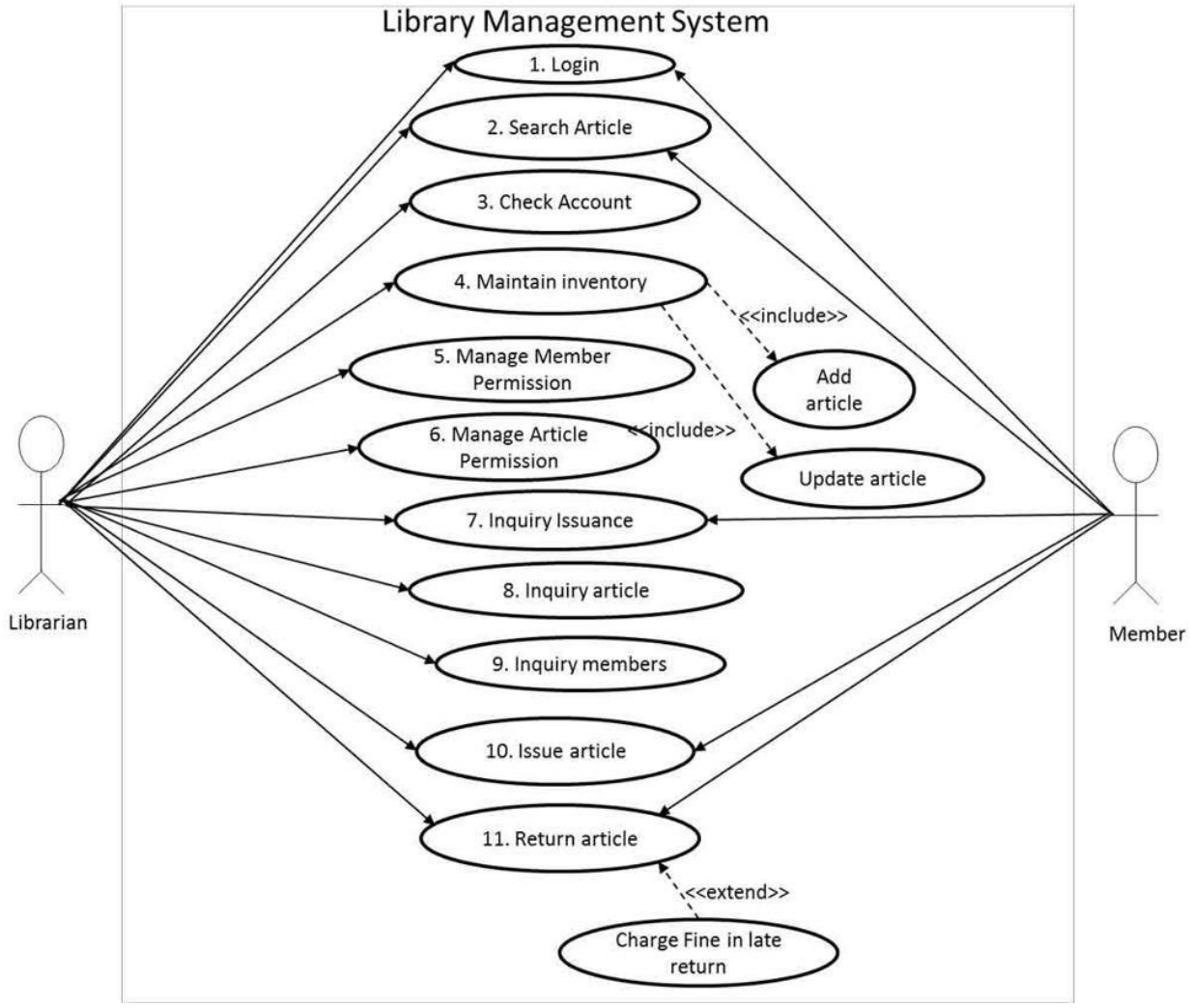
A use case represents a specific function or service provided by the system to an actor.

1. Login: Allows authorized users to access the system.
2. Manage Books: Allows the librarian to add, update and remove book records.
3. Search Books: Allows members and librarians to search by title, author or ISBN.
4. Check Availability: Displays whether a book is available for borrowing.
5. Issue Book: Records the issue of a book to a registered member.
6. Return Book: Records the return of a borrowed book and updates its availability.
7. Calculate Fine: Calculates the fine for books returned after the due date.
8. Manage Members: Allows the librarian to register members and update their details.
9. Generate Reports: Generates reports on books, borrowing, overdue books and fines.
10. Manage Users: Allows the administrator to manage user accounts and access permissions.

4. UML Use Case Diagram

Draw the following diagram in your answer sheet. You can use it as a reference for the actors, system boundary and relationships.

LIBRARY MANAGEMENT SYSTEM — USE CASE DIAGRAM



Reference image for drawing. The diagram below gives the exact proposed structure and connections to reproduce

Note: In a UML use case diagram, draw each actor outside the system boundary, use an oval for every use case, and connect actors to use cases using solid lines. The system boundary is a rectangle labelled Library Management System.

5. Explanation of the Diagram

- The Librarian manages books and members, issues books, processes returns, calculates fines and generates reports.
- The Library Member searches for books, checks availability and views borrowing details.
- The Administrator manages user accounts and access permissions.
- The system boundary represents all the functions included within the Library Management System.
- The association lines represent interactions between actors and the system's use cases.

Conclusion

UML use case modelling helps identify the users of a Library Management System and the functions they perform. It provides a clear representation of system requirements and helps developers understand interactions before implementation.